Import libraries

In [54]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Bank Personal Loan Modelling dataset & explore

Data was obtained from https://www.kaggle.com/teertha/personal-loan-modeling

In [55]:
```
loan_dataset = pd.read_csv('Loan Modelling Thera Bank.csv')
loan_dataset
```

Out[55]:

	ID	Age	Experience	Income	ZIP Code	Family	CCAvg	Education	Mortgage	Perso Lo
0	1	25	1	49	91107	4	1.6	1	0	
1	2	45	19	34	90089	3	1.5	1	0	
2	3	39	15	11	94720	1	1.0	1	0	
3	4	35	9	100	94112	1	2.7	2	0	
4	5	35	8	45	91330	4	1.0	2	0	
...	...	...	...	...	...	...	...	...	...	
4995	4996	29	3	40	92697	1	1.9	3	0	
4996	4997	30	4	15	92037	4	0.4	1	85	
4997	4998	63	39	24	93023	2	0.3	3	0	
4998	4999	65	40	49	90034	3	0.5	2	0	
4999	5000	28	4	83	92612	3	0.8	1	0	

5000 rows × 14 columns

In [56]:
```
loan_dataset.keys()
```

Out[56]:
```
Index(['ID', 'Age', 'Experience', 'Income', 'ZIP Code', 'Family', 'CCAvg',
       'Education', 'Mortgage', 'Personal Loan', 'Securities Account',
       'CD Account', 'Online', 'CreditCard'],
      dtype='object')
```

In [57]:
```
loan_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 14 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   ID                  5000 non-null   int64
 1   Age                 5000 non-null   int64
 2   Experience          5000 non-null   int64
 3   Income              5000 non-null   int64
 4   ZIP Code            5000 non-null   int64
 5   Family              5000 non-null   int64
 6   CCAvg               5000 non-null   float64
 7   Education           5000 non-null   int64
 8   Mortgage            5000 non-null   int64
 9   Personal Loan       5000 non-null   int64
 10  Securities Account  5000 non-null   int64
 11  CD Account          5000 non-null   int64
 12  Online              5000 non-null   int64
 13  CreditCard          5000 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 547.0 KB
```

In [58]:
```
loan_dataset.describe()
```

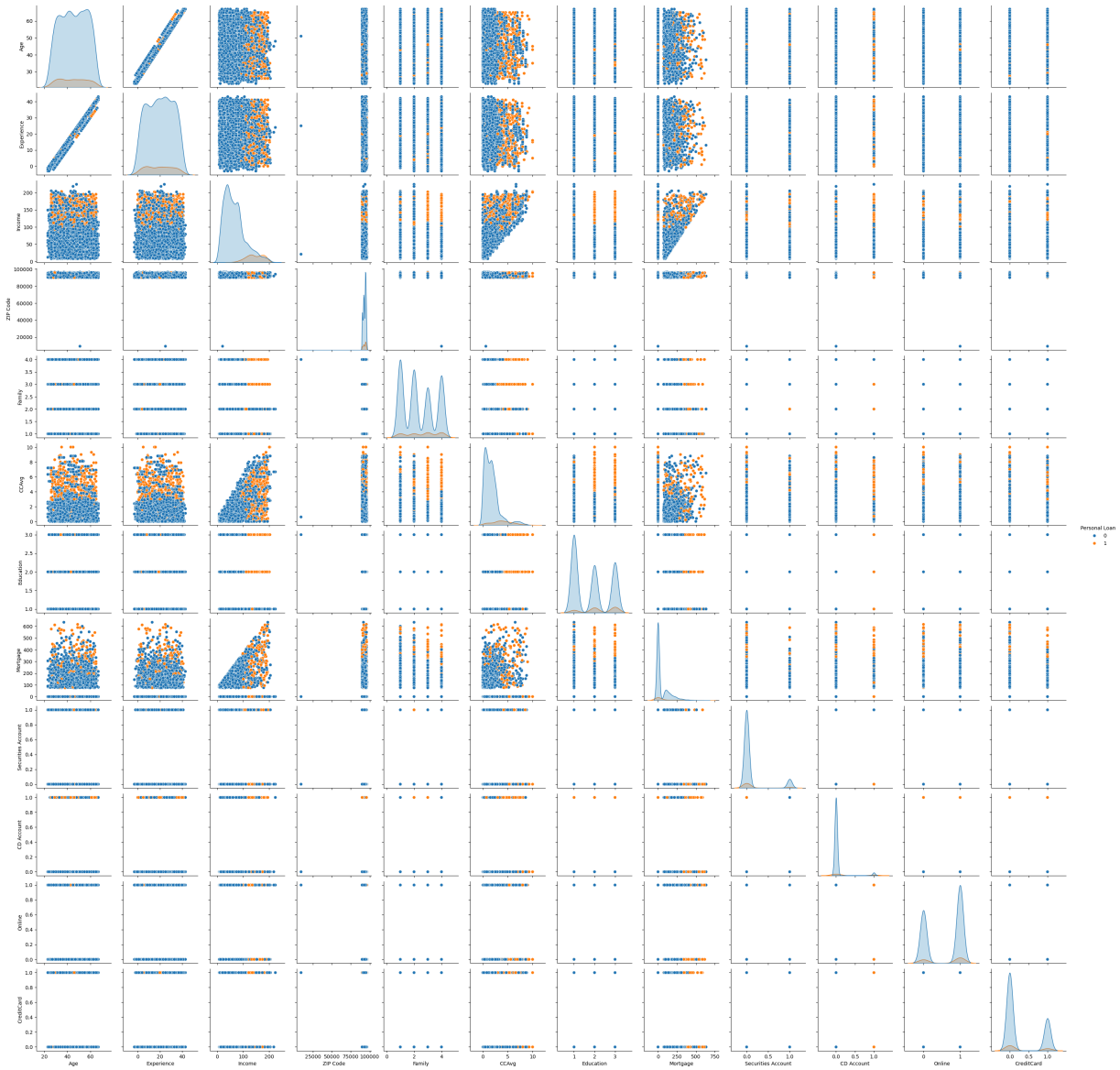
Out[58]:

	ID	Age	Experience	Income	ZIP Code	Family	
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	500
mean	2500.500000	45.338400	20.104600	73.774200	93152.503000	2.396400	
std	1443.520003	11.463166	11.467954	46.033729	2121.852197	1.147663	
min	1.000000	23.000000	-3.000000	8.000000	9307.000000	1.000000	
25%	1250.750000	35.000000	10.000000	39.000000	91911.000000	1.000000	
50%	2500.500000	45.000000	20.000000	64.000000	93437.000000	2.000000	
75%	3750.250000	55.000000	30.000000	98.000000	94608.000000	3.000000	
max	5000.000000	67.000000	43.000000	224.000000	96651.000000	4.000000	1

Data visualization

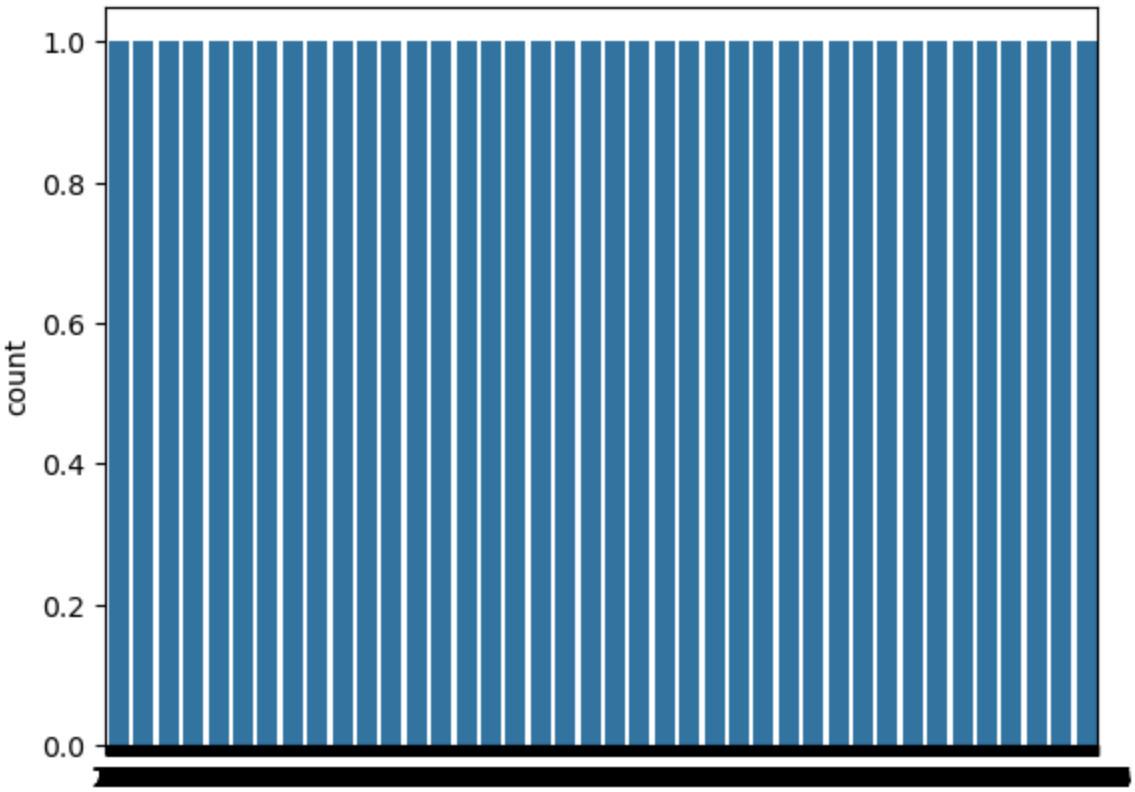
In [59]:
```
sns.pairplot(loan_dataset,
             hue='Personal Loan',
             vars=['Age', 'Experience', 'Income', 'ZIP Code', 'Family', 'CCAvg',
                   'Education', 'Mortgage', 'Securities Account',
                   'CD Account', 'Online', 'CreditCard'])
```

Out[59]: <seaborn.axisgrid.PairGrid at 0x217276702d0>



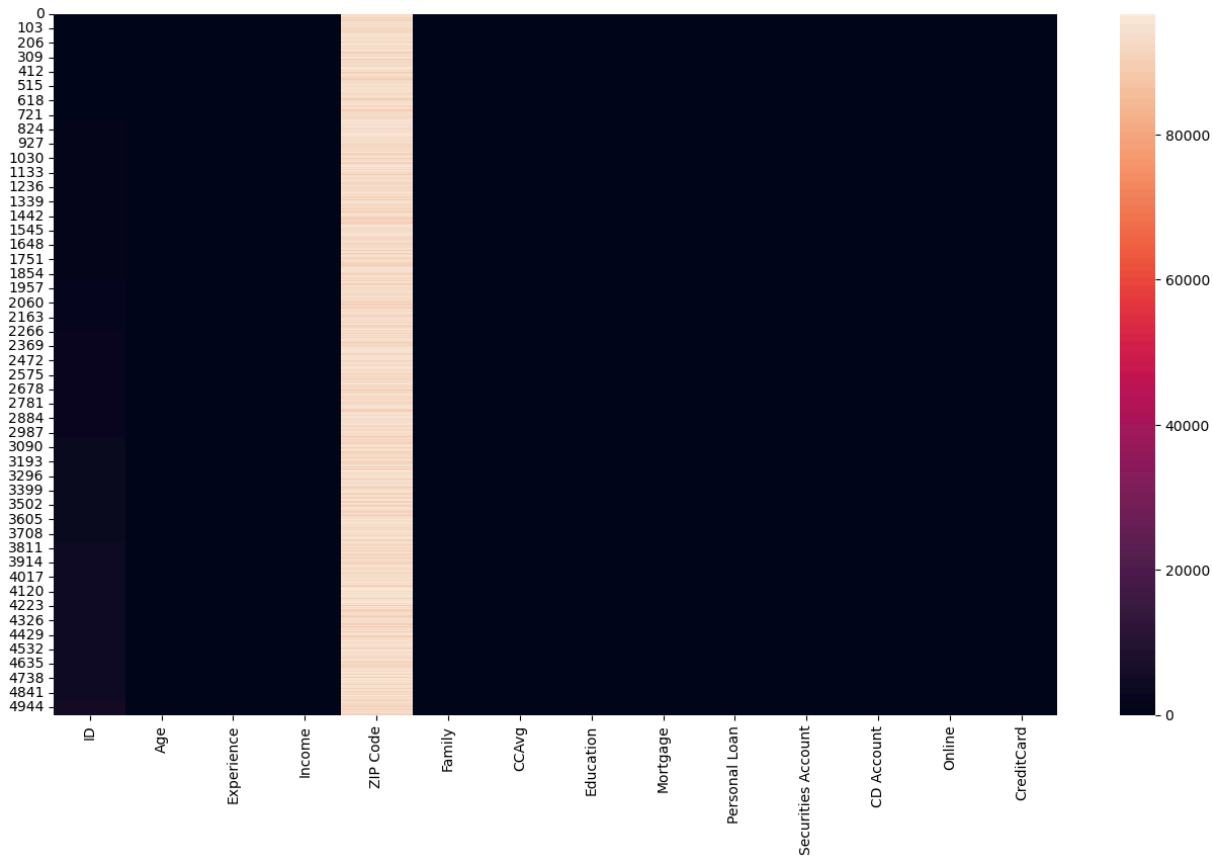
In [60]:
```
sns.countplot(loan_dataset['Personal Loan'])
```

Out[60]: <Axes: ylabel='count'>



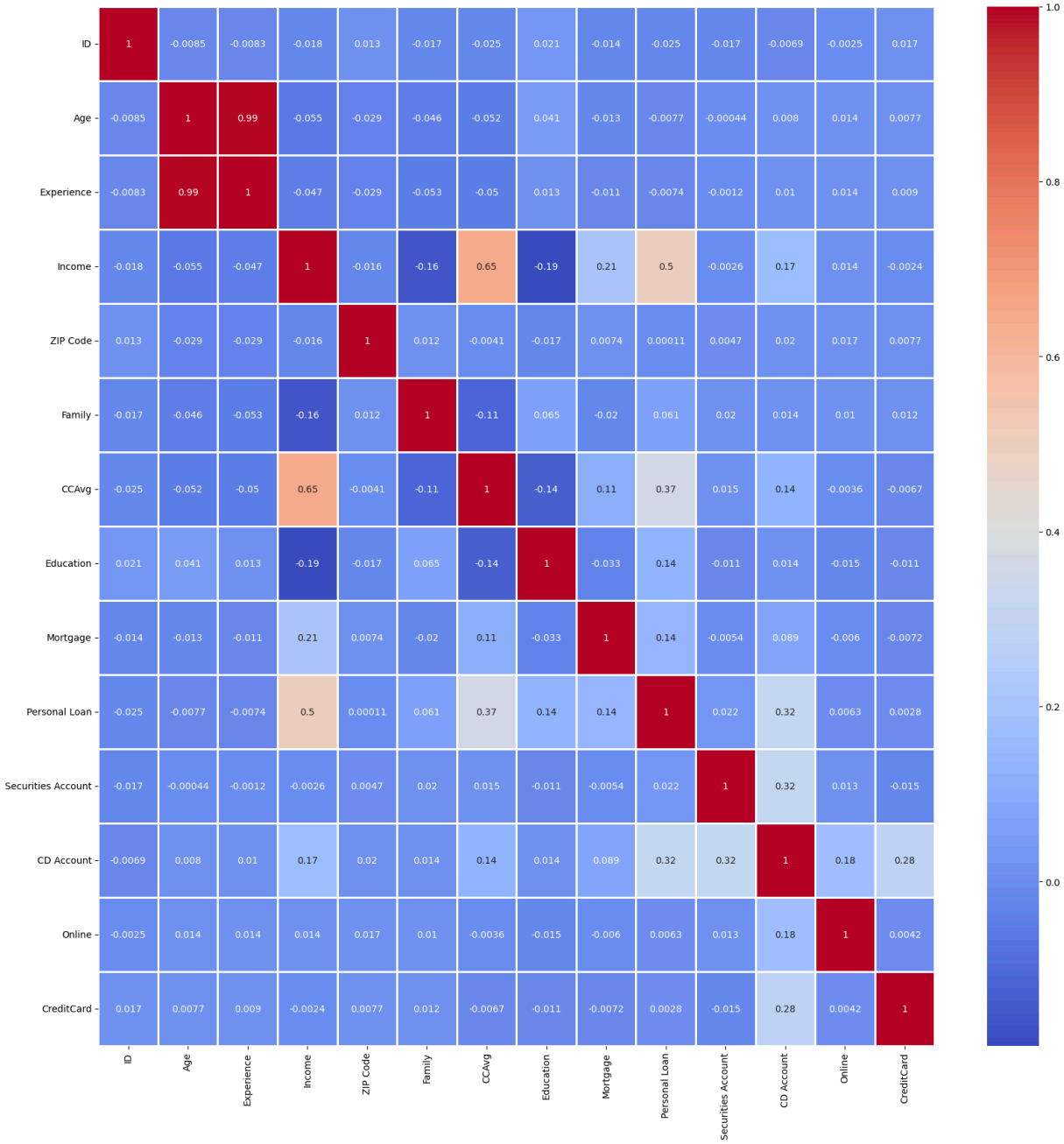
In [61]:
```
plt.figure(figsize=(16,9))
sns.heatmap(loan_dataset)
```

Out[61]: <Axes: >



In [62]:
```
plt.figure(figsize=(20,20))
sns.heatmap(loan_dataset.corr(), annot=True, cmap='coolwarm', linewidths=2)
```

Out[62]: <Axes: >



Data preprocessing

Normalization

In [63]:
```
X = loan_dataset.drop(['Personal Loan', 'ID'], axis=1)
# X = (X-X.mean())/X.var()
X = (X-X.min())/(X.max()-X.min())
X
```

Out[63]:

	Age	Experience	Income	ZIP Code	Family	CCAvg	Education	Mortgage	S
0	0.045455	0.086957	0.189815	0.936527	1.000000	0.16	0.0	0.000000	
1	0.500000	0.478261	0.120370	0.924872	0.666667	0.15	0.0	0.000000	
2	0.363636	0.391304	0.013889	0.977892	0.000000	0.10	0.0	0.000000	
3	0.272727	0.260870	0.425926	0.970931	0.000000	0.27	0.5	0.000000	
4	0.272727	0.239130	0.171296	0.939080	1.000000	0.10	0.5	0.000000	
...	...	...	...	...	...	...	...	...	
4995	0.136364	0.130435	0.148148	0.954731	0.000000	0.19	1.0	0.000000	
4996	0.159091	0.152174	0.032407	0.947174	1.000000	0.04	0.0	0.133858	
4997	0.909091	0.913043	0.074074	0.958463	0.333333	0.03	1.0	0.000000	
4998	0.954545	0.934783	0.189815	0.924242	0.666667	0.05	0.5	0.000000	
4999	0.113636	0.152174	0.347222	0.953758	0.666667	0.08	0.0	0.000000	

5000 rows × 12 columns

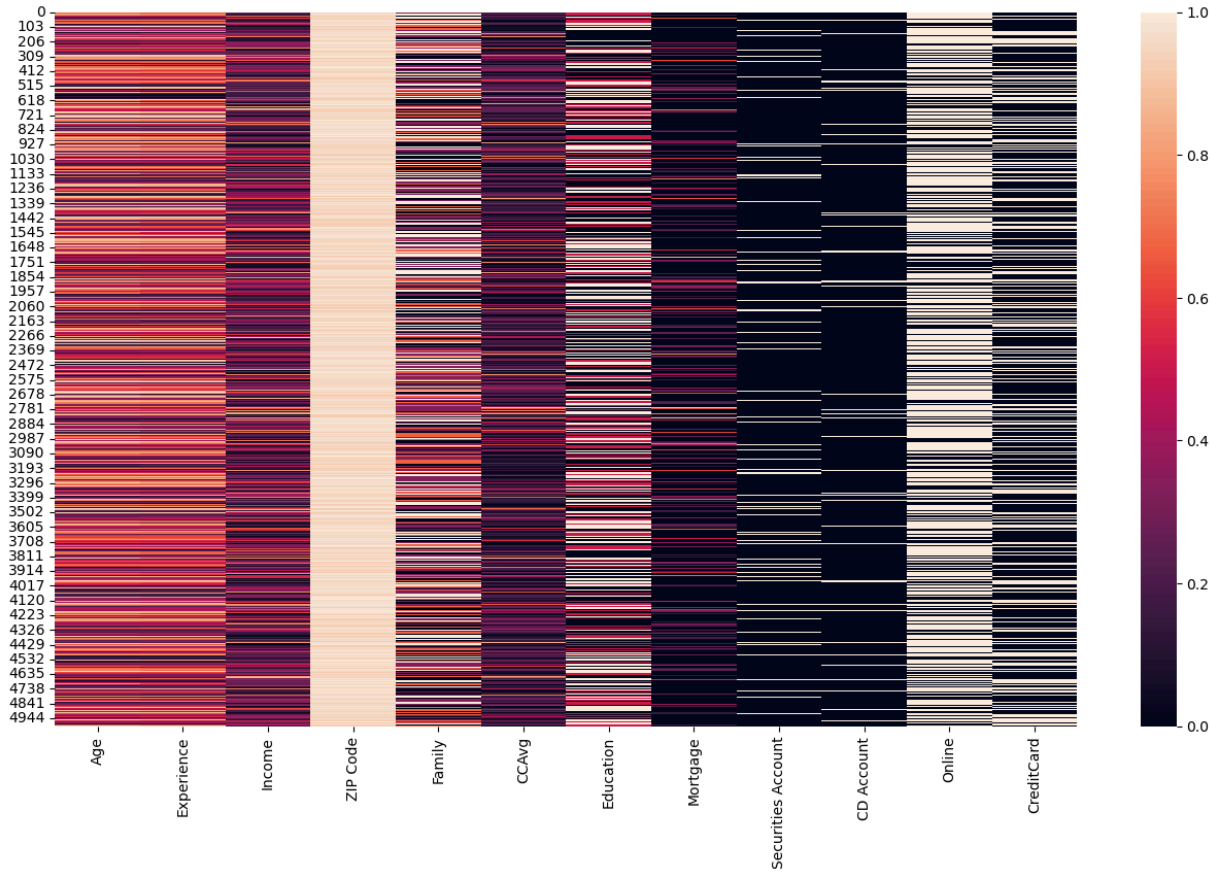
In [64]:
```
X.describe()
```

Out[64]:

	Age	Experience	Income	ZIP Code	Family	CCAvg	Ed
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000
mean	0.507691	0.502274	0.304510	0.959946	0.465467	0.193794	0
std	0.260526	0.249303	0.213119	0.024293	0.382554	0.174766	0
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0
25%	0.272727	0.282609	0.143519	0.945732	0.000000	0.070000	0
50%	0.500000	0.500000	0.259259	0.963203	0.333333	0.150000	0
75%	0.727273	0.717391	0.416667	0.976610	0.666667	0.250000	1
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1

In [65]:
```
plt.figure(figsize=(16,9))
sns.heatmap(X)
```

Out[65]:
```
<Axes: >
```



In [66]:
```
y = loan_dataset['Personal Loan']
y
```

Out[66]:
```
0       0
1       0
2       0
3       0
4       0
       ..
4995    0
4996    0
4997    0
4998    0
4999    0
Name: Personal Loan, Length: 5000, dtype: int64
```

Split dataset into 2 part: training and testing

- Training accounts for 80 %
- Testing accounts for 20 %

In [71]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
print("Training = ", X_train.shape, y_train.shape)
print("Testing = ", X_test.shape, y_test.shape)
```

```
Training =  (4000, 12) (4000,)
Testing =  (1000, 12) (1000,)
```

Personal Loan Modeling

In [72]:
```
from sklearn.metrics import precision_score, recall_score, accuracy_score
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

%matplotlib inline
```

K - Nearest Neightbor Classifier

Classification Problem Using KNN

Objectives:

- To build a KNN model using the sklearn library.
- To apply and understand how to use the KNN model to solve real-world problems (e.g., classification).
- To use Accuracy, Precision, and Recall metrics to evaluate the model's quality.

Problem:

- Given a dataset without labels, how do we determine which label each data belongs to?
- => Build a machine learning model capable of classification.

Data:

- Bank Personal Loan Modeling data
- See also: https://www.kaggle.com/teertha/personal-loan-modeling

Problem:

- Input: 1 sample $X = [x_1, x_2, \ldots x_n]$
- Output: label y is 0 or 1

In [73]:
```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

%matplotlib inline
```

1. KNN

- `KNeighborsClassifier(n_neighbors = 10, metric = 'minkowski', p = 2)`
- n_neighbors = 5
- Euclide p = 2

In [74]:
```
knn_classifier = KNeighborsClassifier(n_neighbors = 10, metric = 'minkowski', p = 2
knn_classifier.fit(X_train, y_train)
```

Out[74]:
```
▾ KNeighborsClassifier
▶ Parameters
```

2. Testing KNN model

Evaluation based on metrics

In [75]:
```
from sklearn.metrics import precision_score, recall_score, accuracy_score

print("Testing...\n")
y_pred_knn = knn_classifier.predict(X_test)
print('Accuracy: ', accuracy_score(y_test, y_pred_knn))
print('Precision: ', precision_score(y_test, y_pred_knn))
print('Recall: ', recall_score(y_test, y_pred_knn))
```

```
Testing...

Accuracy:  0.952
Precision:  0.9166666666666666
Recall:  0.5
```

3. Model selection

Select the number of neighbors

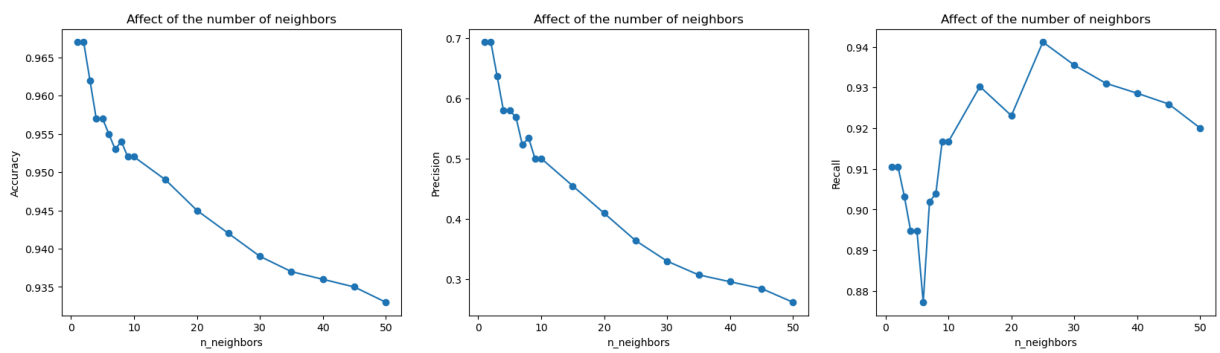
- Change the number of neighbors to find the value that yields the best classification result.

In [82]:
```
print("Evaluation of KNN with different K values")
n_neighbors = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40, 45, 50]
models = []
acc, pre, re = [], [], []
for n in n_neighbors:
    knn_classifier = KNeighborsClassifier(n_neighbors = n, metric = 'minkowski', p
    knn_classifier.fit(X_train, y_train)
    y_pred = knn_classifier.predict(X_test)
    acc.append(accuracy_score(y_pred, y_test))
    pre.append(precision_score(y_pred, y_test))
```

```
    re.append(recall_score(y_pred, y_test))

fig, axs = plt.subplots(1, 3, figsize = (20, 5))

axs[0].plot(n_neighbors, acc, marker = "o")
axs[0].set_xlabel("n_neighbors")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of the number of neighbors")

axs[1].plot(n_neighbors, pre, marker = "o")
axs[1].set_xlabel("n_neighbors")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of the number of neighbors")

axs[2].plot(n_neighbors, re, marker = "o")
axs[2].set_xlabel("n_neighbors")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of the number of neighbors")
```

Evaluation of KNN with different K values

Out[82]: Text(0.5, 1.0, 'Affect of the number of neighbors')



Attribute Selection

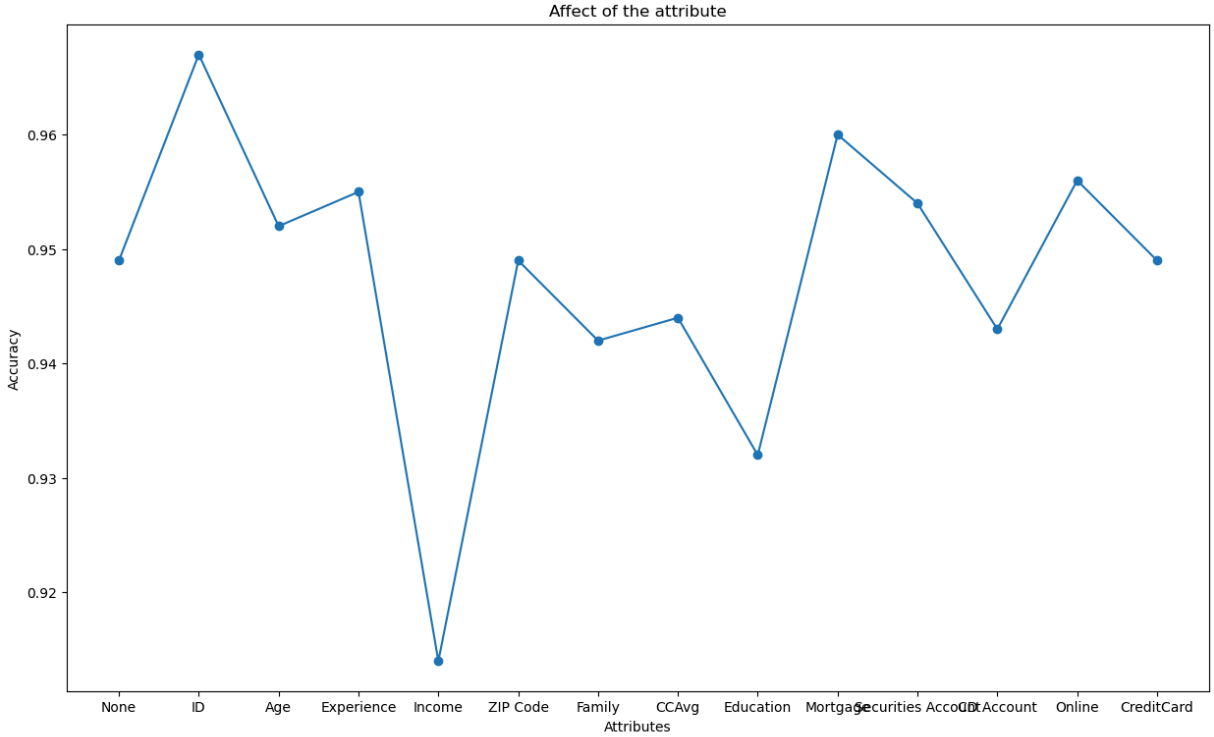
- Attributes: 'ID', 'Age', 'Experience', 'Income', 'ZIP Code', 'Family', 'CCAvg', 'Education', 'Mortgage', 'Personal Loan', 'Securities Account', 'CD Account', 'Online', 'CreditCard'.
- Try removing each attribute from the data to see how it affects the classification results.
- Which attributes should be used to achieve the best classification results?

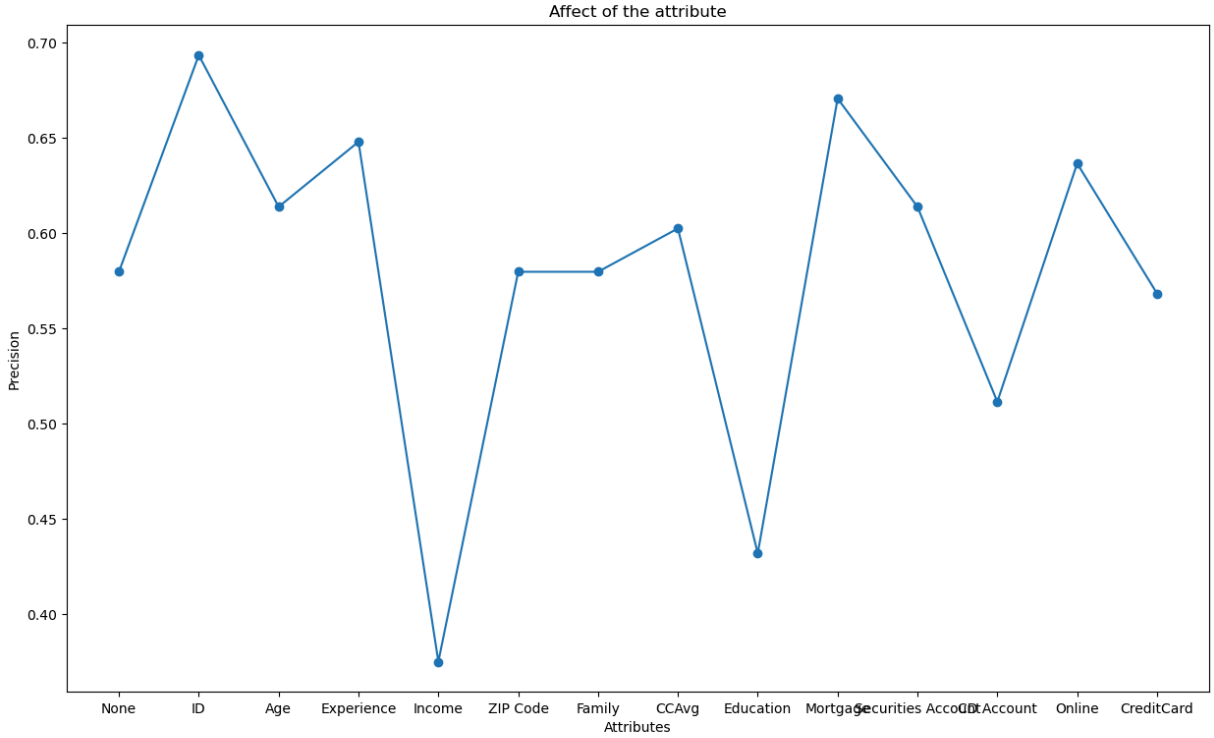
In [92]:
```
features = loan_dataset.drop(['Personal Loan'], axis=1).columns.tolist()
features.insert(0,"None")
acc_f, pre_f, re_f = [], [], []
for feature in features:
    if feature == "None":
        X = loan_dataset.drop(['Personal Loan'], axis=1)
    else:
        X = loan_dataset.drop(feature, axis=1).drop(['Personal Loan'], axis=1)
    X = (X-X.min())/(X.max()-X.min())
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
    knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = 'minkowski', p
    knn_classifier.fit(X_train, y_train)
    y_pred = knn_classifier.predict(X_test)
    acc_f.append(accuracy_score(y_pred, y_test))
```

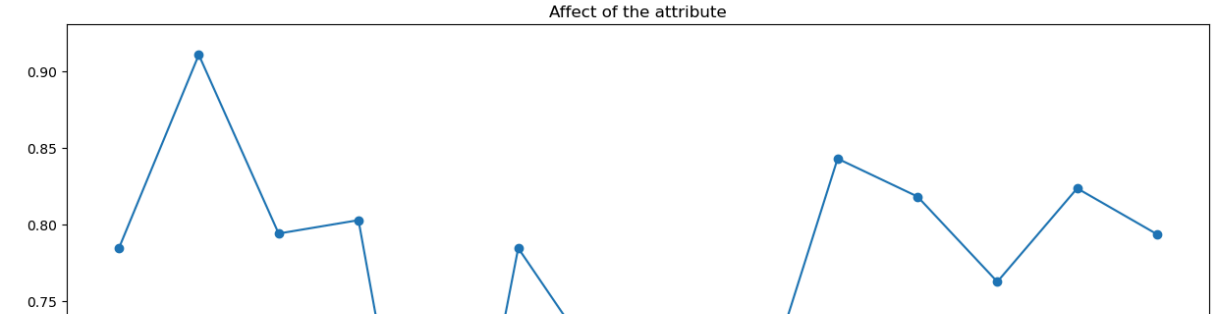
```
    pre_f.append(precision_score(y_pred, y_test))
    re_f.append(recall_score(y_pred, y_test))

fig, axs = plt.subplots(3, 1, figsize = (15, 30))

axs[0].plot(features, acc_f, marker = "o")
axs[0].set_xlabel("Attributes")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of the attribute")

axs[1].plot(features, pre_f, marker = "o")
axs[1].set_xlabel("Attributes")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of the attribute")

axs[2].plot(features, re_f, marker = "o")
axs[2].set_xlabel("Attributes")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of the attribute")
```

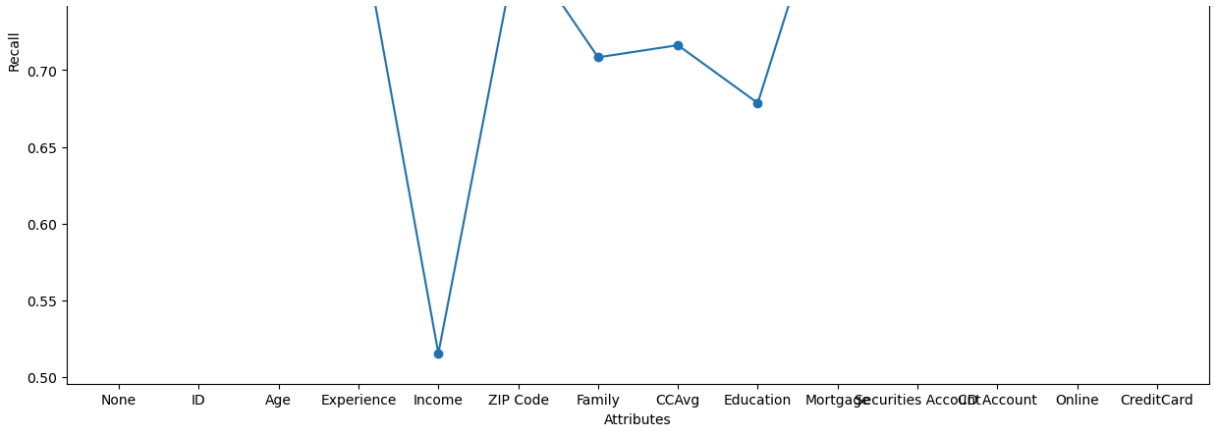
Out[92]:

```
Text(0.5, 1.0, 'Affect of the attribute')
```


Affect of the attribute
Accuracy
0.96
0.95
0.94
0.93
0.92
None
ID
Age
Experience
Income
ZIP Code
Family
CCAvg
Education
Mortgage
Securities Account
CD Account
Online
CreditCard
Attributes


Affect of the attribute
Precision
0.70
0.65
0.60
0.55
0.50
0.45
0.40
None
ID
Age
Experience
Income
ZIP Code
Family
CCAvg
Education
Mortgage
Securities Account
CD Account
Online
CreditCard
Attributes


Affect of the attribute
0.90
0.85
0.80
0.75

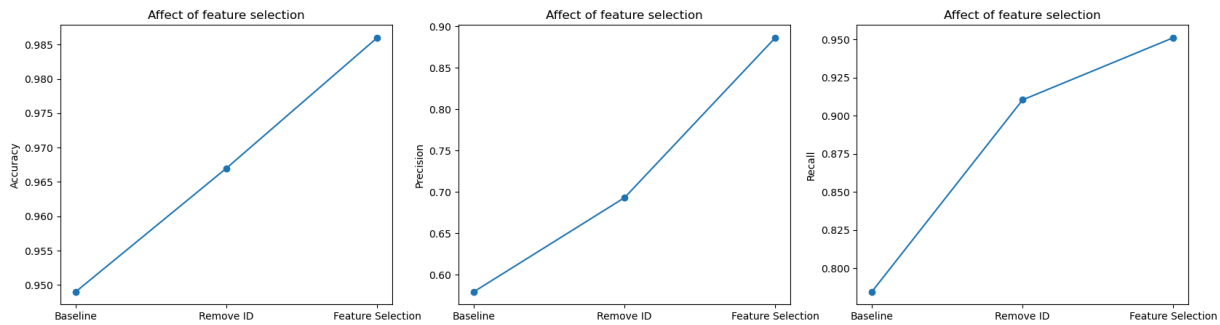


- Nên chọn các features: Income, Education, Family, CD Account, CCAvg

In [95]:
```
acc_new, pre_new, re_new = [], [], []

# Baseline
X = loan_dataset.drop(['Personal Loan'], axis=1)
X = (X-X.min())/(X.max()-X.min())
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = 'minkowski', p = 2,
knn_classifier.fit(X_train, y_train)
y_pred = knn_classifier.predict(X_test)
acc_new.append(accuracy_score(y_pred, y_test))
pre_new.append(precision_score(y_pred,y_test))
re_new.append(recall_score(y_pred, y_test))

# Remove ID
X = loan_dataset.drop(['Personal Loan', 'ID'], axis=1)
X = (X-X.min())/(X.max()-X.min())
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = 'minkowski', p = 2,
knn_classifier.fit(X_train, y_train)
y_pred = knn_classifier.predict(X_test)
acc_new.append(accuracy_score(y_pred, y_test))
pre_new.append(precision_score(y_pred,y_test))
re_new.append(recall_score(y_pred, y_test))

# Feature selection
selected_features = ['Income', 'Education', 'Family', 'CD Account', 'CCAvg']
X = loan_dataset[selected_features]
X = (X-X.min())/(X.max()-X.min())
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = 'minkowski', p = 2,
knn_classifier.fit(X_train, y_train)
y_pred = knn_classifier.predict(X_test)
acc_new.append(accuracy_score(y_pred, y_test))
pre_new.append(precision_score(y_pred,y_test))
re_new.append(recall_score(y_pred, y_test))

method = ["Baseline", "Remove ID", "Feature Selection"]
fig, axs = plt.subplots(1, 3, figsize = (20, 5))
```

```
axs[0].plot(method, acc_new, marker = "o")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of feature selection")

axs[1].plot(method, pre_new, marker = "o")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of feature selection")

axs[2].plot(method, re_new, marker = "o")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of feature selection")
```

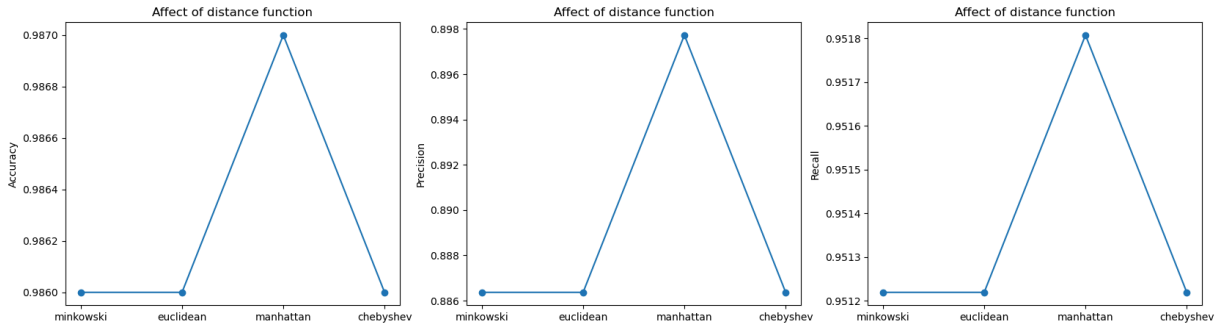
Out[95]: Text(0.5, 1.0, 'Affect of feature selection')



Distance Selection

- Distance function: minkowski, manhattan, euclidean, chebyshev
- Which distance function is best for this problem?

In [96]:

```
metrics = ['minkowski', 'euclidean', 'manhattan', 'chebyshev']
acc_metric, pre_metric, re_metric = [], [], []
for metric in metrics:
    knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = metric, p = 2,
    knn_classifier.fit(X_train, y_train)
    y_pred = knn_classifier.predict(X_test)
    acc_metric.append(accuracy_score(y_pred, y_test))
    pre_metric.append(precision_score(y_pred,y_test))
    re_metric.append(recall_score(y_pred, y_test))

fig, axs = plt.subplots(1, 3, figsize = (20, 5))

axs[0].plot(metrics, acc_metric, marker = "o")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of distance function")

axs[1].plot(metrics, pre_metric, marker = "o")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of distance function")

axs[2].plot(metrics, re_metric, marker = "o")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of distance function")
```

Out[96]: Text(0.5, 1.0, 'Affect of distance function')